

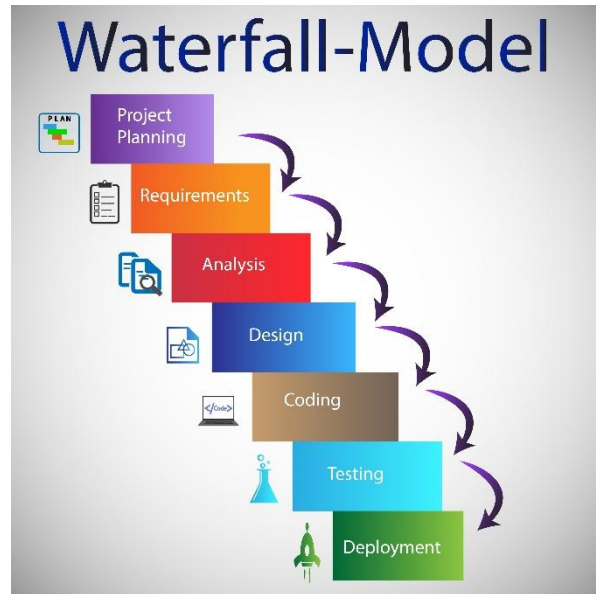
AIM :- Study and compare different software process models and compare them based on cost, simplicity, risk, involvement of user, flexibility, maintenance, integrity, security, re-usability, and requirement.

Parameter	Waterfall	V-Model	Iterative	Spiral	Agile
Cost	Low initial cost as it follows a fixed, linear budget plan.	Moderate cost due to extensive and parallel testing resources.	Can become high because repeated cycles require continuous funding.	Highest cost due to constant risk assessment and prototyping.	Moderate; costs depend on the number of sprints and team size.
Simplicity	Very easy to understand as it follows a "one-step-at-a-time" flow.	Simple but disciplined, requiring strict mapping of tests to design.	Moderately complex; requires managing multiple versions of the product.	Most complex; requires highly skilled experts for risk analysis.	Moderate; easy to start but requires cultural shift and discipline.
Risk	High risk; errors are often discovered only during the final stages.	Moderate; early test planning reduces some risk but late changes are hard.	Reduced risk as each version provides a chance to fix major flaws.	Lowest risk; every cycle starts with identifying and fixing potential failures.	Low risk; frequent deliveries ensure the project stays on track.
User Involvement	Low; the user is usually only involved at the very start and very end.	Moderate; users help define requirements and perform acceptance testing.	High; users provide feedback on each successive version produced.	Moderate; users are involved during the evaluation phase of each loop.	Highest; the customer is a constant partner throughout the process.
Flexibility	Rigid; once a phase is	Rigid; it follows a	Flexible; features	Highly flexible;	Most Flexible;

Parameter	Waterfall	V-Model	Iterative	Spiral	Agile
	finished, going back to change it is difficult.	strict sequence that discourages mid-way changes.	can be adjusted or added in the next iteration.	the model is designed to evolve as the project grows.	welcomes changing requirements even late in development.
Maintenance	Difficult because the entire system is delivered at once as a "black box."	Easier than Waterfall because testing documentation is very thorough.	Manageable since the system is built and maintained in chunks.	Manageable; though complex, the history of risk cycles helps tracking.	Easiest; continuous integration makes ongoing updates part of the flow.
Requirement	Must be clear and fixed before any development starts.	Must be precise and stable to allow for early test design.	Can be partially known and refined as iterations progress.	Often complex, ambiguous, or highly innovative at the start.	Dynamic and expected to change frequently throughout the project.
Example	Static Website: Building a basic 5-page site for a local business with a fixed scope.	Medical Device: Software for a heart monitor where every code line must be verified.	Search Engine: Releasing a text search first, then adding image and video search later.	Defense Systems: Large-scale military projects like a fighter jet flight control system.	Mobile Apps: Developing a social media app where features change based on user trends.

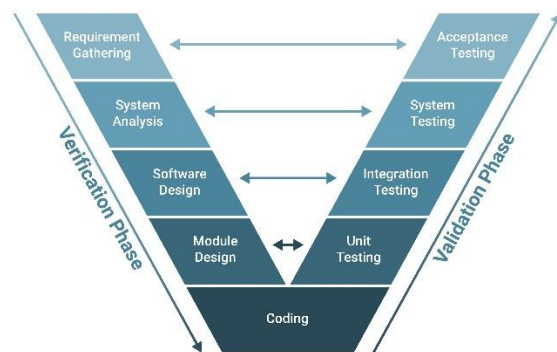
1) Waterfall Model

- This model operates on a strictly sequential basis, requiring the total completion of one stage before the subsequent one can begin.
- The process initiates with a thorough analysis of requirements and concludes only after the final maintenance phase is reached.
- Due to its rigid structure, revisiting a previous stage to implement modifications is highly challenging and often costly.
- Its straightforward design makes the development lifecycle very simple to comprehend and oversee.
- It is most effective for minor projects where the objectives are fixed, well-documented, and unlikely to shift.



2) V-Model

- Functioning as an evolved version of the Waterfall approach, this model places a heavy emphasis on integrating quality checks throughout the entire lifecycle.
- It operates on a symmetrical structure where every single stage of development is directly paired with a specific, corresponding testing level.
- By organizing the testing strategy during the early design stages, the model significantly enhances the overall reliability and quality of the final product.
- Despite the parallel testing, the framework remains relatively inflexible, making it hard to incorporate new requirements once the build is underway.

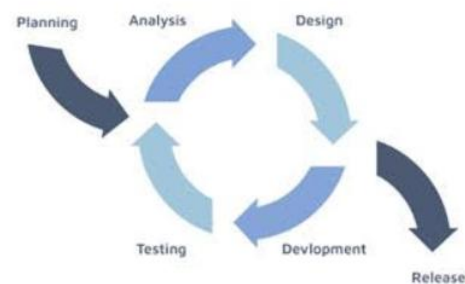


- This model is exceptionally well-suited for safety-critical industries, such as medical technology or embedded aerospace systems, where precision is vital.

3) Iterative Model

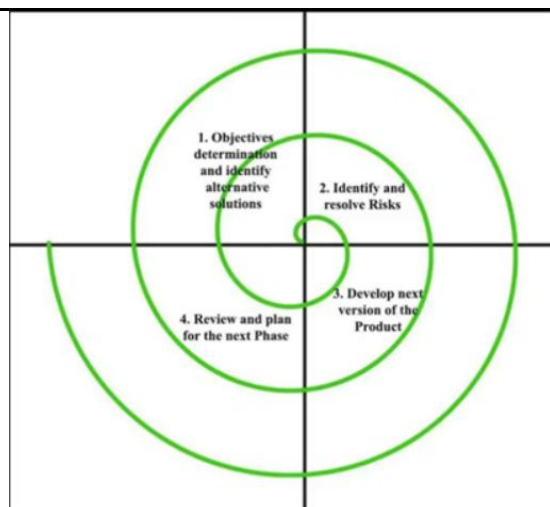
- This framework focuses on building the software through a series of repeating, manageable cycles rather than attempting a single, full release.
- Every individual cycle concludes with the delivery of a functional, executable version of the software, allowing for incremental progress.
- The system is continuously refined and enhanced based on user evaluations and feedback gathered at the end of each iteration.
- Project security and stability are improved because potential flaws and technical risks are identified and addressed early in the process.
- This approach is highly effective for projects where the initial requirements are only partially understood and need to evolve over time.

Iterative Development Model



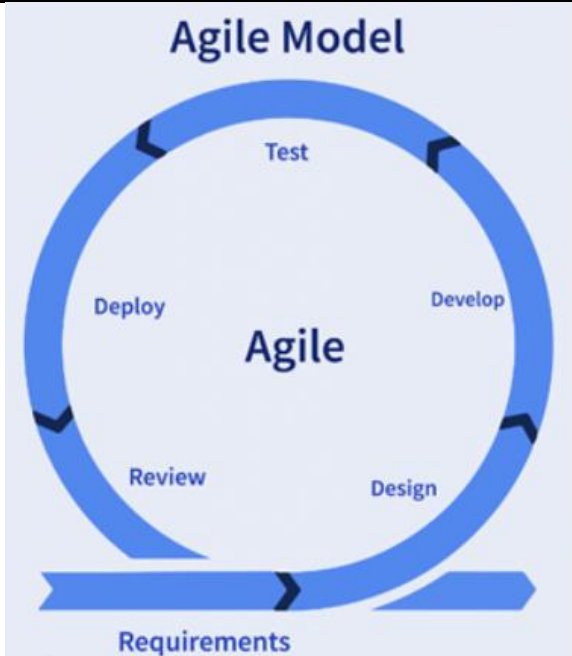
4) Spiral Model

- This framework integrates the repetitive nature of iterative development with a deep, systematic focus on risk management.
- The process advances through continuous loops, where each rotation involves four distinct stages: planning, evaluating risks, engineering, and assessing the results.
- Potential failures are proactively identified and mitigated at the start of



every cycle, ensuring high project stability. • Due to its intensive nature, this model is inherently complex to manage and requires a significant financial investment. • It is the preferred choice for massive, high-stakes, or mission-critical initiatives where avoiding error is the top priority.	
---	--

5) Agile Model

• This model utilizes an incremental and highly adaptive strategy, focusing on rapid delivery and flexibility. • The entire development process is partitioned into brief, time-boxed units known as "sprints," typically lasting a few weeks each. • Success is driven by constant collaboration, with the customer remaining actively involved and providing feedback throughout the lifecycle. • The framework is designed to welcome modifications, allowing for requirement changes to be integrated seamlessly even during late stages of production. • It is ideally suited for dynamic, fast-paced environments, such as the development of mobile applications and modern web platforms.	 The diagram illustrates the Agile Model as a continuous cycle. At the center is the word "Agile". Surrounding it is a circular path with five stages: Test (top), Develop (top-right), Design (bottom-right), Review (bottom-left), and Deploy (top-left). Arrows indicate a clockwise flow between these stages. Below the circular path, a horizontal bar labeled "Requirements" has an arrow pointing right, indicating its integration into the cycle.
---	---